Reducing WITT Model Storage by Tabulating Scalar SNR Gates

Author Name
Institution / Department

Abstract

We reduce the deployment storage of the wireless image transmission transformer (WITT) by tabulating the networks that condition it on signal-to-noise ratio (SNR). Fourteen scalar-input networks account for 4,150,272 learned parameters in our configuration. We replace them after training with 13-node FP32 tables and linear interpolation, retaining the image-dependent computation and channel-use ratio. For one locally trained checkpoint on CIFAR-10 with an additive white Gaussian noise (AWGN) channel, the complete serialized deployment file decreases from 54,987,545 to 38,644,489 bytes, a 29.72% reduction. Validation determines the table choice before a frozen evaluation on 10,000 test images, eight predeclared SNRs, and three paired noise realizations. The maximum signed decrease in peak signal-to-noise ratio (PSNR) is 0.000186 dB. Five test conditions are stored knots and three exercise interpolation. Dense validation and a nearest-neighbor comparison characterize the storage–reconstruction tradeoff.

1. Introduction

Learned joint source–channel coding (JSCC) maps images to channel symbols and reconstructs them after noisy transmission. Its encoder and decoder jointly determine reconstruction quality under a bandwidth and power budget [1]. SNR-adaptive systems condition this mapping on channel quality, allowing one trained model to operate at several SNRs [5, 6]. Deploying such a system also requires storing its conditioning networks, even when those networks receive a single scalar input.

WITT provides a concrete opportunity to examine this cost. In the author implementation used here, seven encoder gates and seven decoder gates each map scalar SNR to a 384-dimensional vector. These gates are independent of the transmitted image. Their outputs can therefore be evaluated before deployment and stored at selected SNR knots. Runtime interpolation then replaces the learned scalar-to-vector maps. The central question is whether removing their parameters can yield a useful reduction in complete model-file size while preserving reconstruction quality between the stored knots.

We train a local model using the WITT authors' components and compile its scalar gates after training. A 13-node table removes 4,150,272 learned parameters and reduces the serialized model file by 29.72%. With the checkpoint and deployment choice fixed before testing, its maximum signed PSNR decrease is 0.000186 dB across eight predeclared CIFAR-10/AWGN test conditions. Five of those SNRs coincide with table knots; the remaining three exercise interpolation.

The study contributes a reproducible intervention on WITT's scalar-only gates and an evaluation that separates checkpoint selection, table selection, reload checks, and final testing. In the historical pilot, equal-size five-knot tables have maximum losses of 0.009071 dB with linear interpolation and 0.129533 dB with nearest lookup, motivating the interpolation rule. A shared-SNR control separates repeated scalar computation from parameter removal, and dense validation examines fractional SNRs on a later checkpoint. Together these measurements characterize the storage–reconstruction tradeoff.

2. Related Work

Learned wireless image transmission. DeepJSCC learns a direct mapping between image pixels and complex channel inputs [1]. By optimizing reconstruction through a simulated channel, it couples source representation with resilience to transmission noise. Attention-based adaptive JSCC incorporates channel quality into feature processing [5]. WITT introduces a transformer architecture and SNR-dependent feature modulation for wireless image transmission [6]. SwinJSCC further develops Swin-transformer-based JSCC with channel and rate adaptation [7]. These studies motivate conditioning the encoder and decoder on communication constraints. Our experiment uses the original WITT components to isolate the storage cost of scalar SNR conditioning at a fixed channel-use ratio.

Lookup tables for neural deployment. Evaluating a learned function in advance and interpolating stored outputs is an established deployment strategy. Jo and Kim trans-

fer learned image mappings into lookup tables for practical single-image super-resolution [2]. The dimensionality of the table input is central to its storage cost: a multidimensional image neighborhood and a single channel-quality scalar lead to different table designs. Here, the tabulated input is only SNR. The table stores outputs of WITT's trained conditioning networks, while image-dependent feature transformations remain in place. The empirical contribution is the measured file-size and reconstruction tradeoff for this intervention, together with a test protocol that checks interpolation rather than only stored knots.

3. Model and Post-Training Compilation

3.1. Author-component reproduction

For an image $x \in [0,1]^{3\times32\times32}$, let f_θ and g_ϕ denote the WITT encoder and decoder. Both receive SNR s in decibels. The encoder produces real-valued latent features that the author channel implementation packs into complex symbols. After power normalization, the simulated channel adds independent real and imaginary Gaussian noise with standard deviation

$$\sigma(s) = \sqrt{1/(2\,10^{s/10})}. \tag{1}$$

The implementation restores the latent scale after transmission and passes the result to the decoder. Its normalization averages power over the batch; paired comparisons therefore preserve batch membership and order.

We use the author's CIFAR-10 configuration with latent width $C = 32$, two encoder stages of depths $[2, 4]$, embedding dimensions $[128, 256]$, and window size 2. The decoder reverses these stages. The channel bandwidth ratio (CBR) is $1/3$, defined as complex channel uses divided by real source samples. The original deployment has 13,715,056 learned parameters. The local training wrapper imports the author's model and channel components without modifying their source.

3.2. Scalar gate tabulation

For each of the 14 gates, let $h_j(s) \in \mathbb{R}^{384}$ be its trained scalar-to-vector map. Each gate multiplies image-derived hidden features elementwise in a feature-modulation branch. A final image-dependent projection and sigmoid produce weights that rescale the encoder or decoder features. Tabulation replaces only h_j, retaining this image-dependent branch. Each scalar map is a multilayer perceptron (MLP) with dimensions $1 \to 384 \to 384 \to 384$, hidden ReLU activations, and a final sigmoid. Including biases, it contains

$$(384 + 384) + 2(384^2 + 384) = 296{,}448 \tag{2}$$

parameters. The 14 maps together contain 4,150,272 parameters.

Deployment	Parameters	File bytes
Original	13,715,056	54,987,545
Linear, 5 knots	9,564,784	38,472,457
Linear, 13 knots	9,564,784	38,644,489

Table 1. Current checkpoint (21,500 updates): complete inference-file sizes and learned-parameter counts. Table buffers are additional stored values, rather than learned parameters.

For K uniformly spaced knots $s_k \in [1, 13]$ dB, we evaluate $v_{j,k} = h_j(s_k)$ in FP32 after training. At runtime, for $s \in [s_k, s_{k+1}]$, we use

$$\widetilde{h}_j(s) = (1-\alpha)v_{j,k} + \alpha v_{j,k+1}, \qquad \alpha = \frac{s - s_k}{s_{k+1} - s_k}. \tag{3}$$

The main deployment uses $K = 13$, giving integer-dB knots. Both the knots and gate outputs are stored as buffers. Their total is $14 \times 13 \times (384 + 1) = 70{,}070$ FP32 values, or 280,280 raw bytes. The retained learned-parameter count is 9,564,784. All image-dependent feature transformations, the Swin backbone, the channel implementation, and CBR remain unchanged. Compilation requires no additional training.

We measure storage from the complete serialized inference files, including metadata, buffers, and serialization overhead. These files exclude optimizer state. We separately check that 223 retained non-gate tensors are unchanged and compare a freshly compiled deployment with the same deployment loaded from its saved file. The recorded maximum output difference for this reload check is zero. Original-versus-table reconstruction differences are measured separately.

4. Experimental Protocol

Data and training. CIFAR-10 contains 50,000 training and 10,000 test images [3]. A fixed permutation with seed 20261107 partitions the training images into 45,000 training and 5,000 validation examples. Inputs are scaled to $[0, 1]$ and horizontally flipped during training with probability 0.5. We use Adam with learning rate 10^{-4}, batch size 128, and one training seed, 42. The optimized distortion is mean squared error (MSE) on the author's 0–255 scale:

$$\mathcal{L} = \frac{255^2}{3BHW} \sum_{b=1}^{B} \|x_b - \mathrm{clip}(\hat{x}_b, 0, 1)\|_2^2, \tag{4}$$

where B is batch size and $H = W = 32$. All modules are constructed at initialization. The first 2,000 updates use the non-adaptive forward path at 7 dB, bypassing the modulation branch. We then activate that branch in the same model instance and sample SNR uniformly from

$\{1, 4, 7, 10, 13\}$ dB. Backbone weights continue from pretraining; previously bypassed modules retain their seeded constructor initialization until this transition. No new parameters are created at the boundary, where Adam is reset. Training segments preserve optimizer, sampler, and random-number-generator states.

Every 500 updates, the adaptive model is evaluated at 1, 7, and 13 dB using noise seed 9917 plus $100s$. Selection minimizes the arithmetic mean of the three MSEs, retaining the earlier checkpoint on ties. Among 41 scheduled evaluations, the selected checkpoint occurs at 21,500 total updates. Cutoff-only validations are excluded. All current deployment variants use this checkpoint. Computation uses 32-bit floating point (FP32), PyTorch 2.9.1 with CUDA 12.8, and an RTX 5070 Laptop GPU.

Validation before final testing. A historical pilot uses the 8,000-update checkpoint, 1,000 validation images, 13 fixed SNRs, and three paired channel-noise realizations. Four pilot points (2.25, 5.25, 8.25, and 11.25 dB) are off the 13-node table's knots. It compares 3-, 5-, 9-, and 13-node linear tables, a 5-node nearest-neighbor table, and a control that computes each original SNR MLP once per batch and broadcasts its output. The primary 13-node design is fixed after reviewing this pilot.

On the 21,500-update checkpoint, P002 compares the original model with 5- and 13-node linear tables using 1,000 validation images at 32 SNRs. These include every half-decibel point from 1 to 13 dB and seven additional fractional stress points. P003 compares the original and primary 13-node deployment on all 5,000 validation images at the eight SNRs subsequently used for testing. Each validation uses three paired noise realizations. Admission requires maximum signed PSNR loss at most 0.1 dB, at least 25% reduction in serialized bytes, and a successful save-and-reload check.

Both validations pass before the checkpoint, deployment files, and P004 test protocol are frozen. P004 evaluates all 10,000 official test images at $\{1, 1.5, 2.375, 3, 7, 12, 12.5, 13\}$ dB using three new paired noise realizations. Deployment validations P002 and P003 use noise seed base 51001; P004 uses 91001. Testing makes no further selection of knots or model parameters. Its reload check uses a deterministic synthetic ramp, separate from the official test images.

Metrics and conditional uncertainty. For N distinct images at each SNR, we clip reconstructed pixels to $[0, 1]$ and compute an MSE $e_{i,r}$ over the 3,072 scalar RGB channel values of image i in noise realization r. Reported PSNR is

$$P = -10 \log_{10} \left(\frac{1}{3N} \sum_{r=1}^{3} \sum_{i=1}^{N} e_{i,r} \right). \tag{5}$$

This is PSNR from pooled MSE, rather than the mean of per-image PSNRs. Signed loss is $\Delta = P_{\text{original}} - P_{\text{table}}$, so a negative value denotes a small improvement for the table.

For P002–P004, at each SNR, 2,000 paired bootstrap draws resample original batch IDs jointly across both methods and all three noise realizations. Each draw pools the recorded error sums and image counts, correctly weighting the shorter last batch. The full validation has 40 batch clusters (last batch: 8 images), and testing has 79 (last batch: 16). The 2.5th and 97.5th percentiles form pointwise descriptive intervals conditional on the checkpoint, fixed batch members, and recorded noises. The three noise realizations are repetitions of the channel simulation, rather than independent training seeds.

5. Results

Deployment storage. Table 1 reports actual saved inference-file sizes. The main 13-node deployment occupies 38,644,489 bytes versus 54,987,545 for the original model, a 29.72% reduction. Five nodes save a further 172,032 bytes, only a small additional reduction relative to the full original file. The measured storage improvement follows from removing the gate parameters while retaining compact FP32 lookup buffers.

Interpolation and table density. On dense P002 validation, the 13-node table's maximum signed loss is 0.000454 dB at 2.25 dB, whereas the 5-node table reaches 0.015405 dB at 2.5 dB (Fig. 1). Both pass the predeclared tolerance. The 13-node choice offers the smaller observed loss for a modest file-size increase. On all 5,000 validation images in P003, its maximum signed loss is 0.000264 dB at 2.375 dB.

Frozen official test. Table 2 gives absolute reconstruction PSNR and the paired difference on the frozen test grid. The maximum signed decrease is $+0.186$ millidecibels (mdB) at 2.375 dB, with descriptive interval $[-0.041, +0.412]$ mdB. The maximum absolute change is 0.338 mdB at 1.5 dB, a gain for the table. These are different summaries of the same paired results.

Five test conditions are integer-dB table knots and show differences below 3×10^{-8} dB in magnitude. The three fractional conditions, 1.5, 2.375, and 12.5 dB, exercise interpolation. Figure 2 displays these differences in millidecibels so that their scale and conditional intervals remain visible. The fractional conditions include both positive and negative differences.

Historical controls and failure case. Table 3 retains all pilot variants on their original 8,000-update checkpoint and 13-point validation grid. At the same five knots and file size of 38,472,457 bytes, linear interpolation has maximum

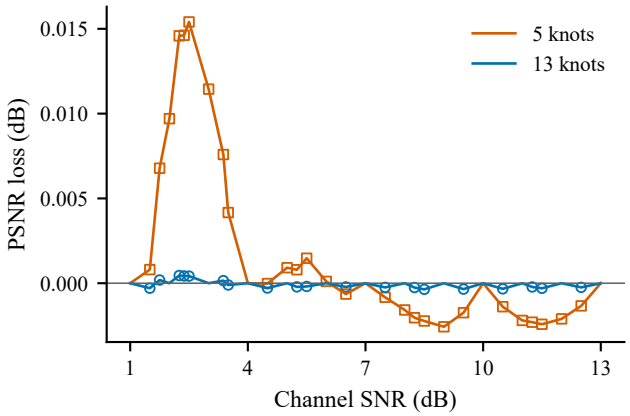


Figure 1. Dense validation (P002): original PSNR minus table PSNR at 32 fixed SNRs, with 1,000 images and three paired noise realizations per condition. Lines guide the eye between measured points; open markers identify non-knot conditions for each table. Five-node knots are 1, 4, 7, 10, and 13 dB; 13-node knots are every integer from 1 to 13 dB. Both variants use checkpoint 21,500.

SNR (dB)	Original (dB)	13 knots (dB)	Loss (mdB)	95% interval (mdB)
1	27.09260	27.09260	0.000	[0.000, 0.000]
1.5	27.53395	27.53429	-0.338	[-0.479, -0.188]
2.375	28.27304	28.27285	+0.186	[-0.041, 0.412]
3	28.78591	28.78591	0.000	[0.000, 0.000]
7	31.84959	31.84959	0.000	[0.000, 0.000]
12	34.87276	34.87276	0.000	[0.000, 0.000]
12.5	35.11328	35.11354	-0.258	[-0.269, -0.246]
13	35.33646	35.33646	0.000	[0.000, 0.000]

Table 2. P004 official test: 10,000 images and three paired noises at each SNR. PSNR is computed from pooled MSE. Loss is original minus 13-node PSNR, in millidecibels (mdB; $1\,\mathrm{mdB} = 10^{-3}$ dB). Intervals are conditional, pointwise, and descriptive (Sec. 4). Values, including near-zero knot differences, are rounded for display.

loss 0.009071 dB, versus 0.129533 dB for nearest-neighbor lookup. This comparison isolates the interpolation rule. Reducing the linear table to three knots raises its maximum loss to 0.093756 dB, illustrating the density tradeoff. The shared-SNR control preserves the original MLP parameters and shows a near-zero reconstruction difference, separating repeated scalar computation from parameter removal. Its displayed loss rounds to zero. The table values describe this historical checkpoint and grid; the dense current-checkpoint results above are a distinct experiment.

6. Discussion and Conclusion

Tabulating WITT's scalar SNR gates removes a substantial parameter block while preserving its image-dependent processing. For the selected CIFAR-10/AWGN checkpoint, a 13-node deployment reduces complete serialized storage by 29.72%, with maximum signed PSNR decrease 0.000186 dB on the frozen eight-point official test. Dense validation and a nearest-neighbor comparison show why evaluating off-knot conditions is essential when compiling a conditional neural network into tables.

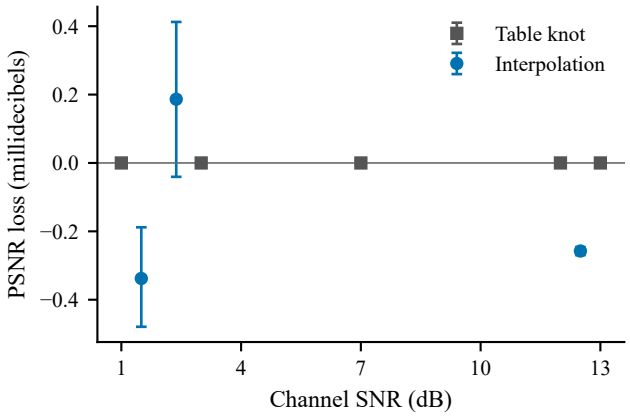


Figure 2. Frozen P004 test differences and conditional pointwise 95% descriptive intervals from 2,000 paired batch-cluster bootstrap draws. Each condition uses 10,000 images, three noises, and 79 batch clusters. Squares mark table knots; circles mark interpolation. Positive loss means lower PSNR for the table.

Method	File bytes	Max. loss (dB)
Original	54,987,481	0.000000
Shared-SNR MLP	54,989,849	0.000000
Linear, 3 knots	38,429,449	0.093756
Linear, 5 knots	38,472,457	0.009071
Linear, 9 knots	38,558,473	0.000389
Linear, 13 knots	38,644,489	0.000258
Nearest, 5 knots	38,472,457	0.129533

Table 3. Historical P001 pilot: 8,000-update checkpoint, 1,000 validation images, 13 SNR conditions, three paired noises. Maximum signed loss is taken over the full recorded grid. This checkpoint is separate from Tables 1 and 2.

Limitations and next steps. The evidence covers one architecture, one training trajectory, CIFAR-10 images, simulated AWGN, and CBR $1/3$. Modern libraries, a validation holdout, and a finite update budget differ from the author's training protocol; numerical parity with the published WITT results and convergence remain unestablished. The two checkpoints share one training trajectory. The 13 knots cover 1–13 dB; the implementation clamps outside this range, and domain-exterior behavior is untested. Eight test points and conditional intervals support pointwise conclusions rather than a continuous-SNR guarantee. PSNR measures reconstruction, leaving task-level semantic utility

and real-link performance for separate evaluation. Fixed-order pilot timings at 7 dB were mixed: 13 nodes versus original took 40.4 versus 36.2 ms at batch size 1 and 43.7 versus 46.3 ms at batch size 128 (medians of 30 measured forwards after 10 warmups). These observations establish no consistent speed advantage. Further work should examine additional seeds and architectures, rate adaptation, and device-level storage and latency together.

References

[1] Eirina Bourtsoulatze, David Burth Kurka, and Deniz Gündüz. Deep joint source-channel coding for wireless image transmission. *IEEE Transactions on Cognitive Communications and Networking*, 5(3):567–579, 2019.

[2] Younghyun Jo and Seon Joo Kim. Practical single-image super-resolution using look-up table. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 691–700. IEEE, 2021.

[3] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[4] wanshuiyin. Auto-claude-code-research-in-sleep (ARIS). Software repository, 2026. Commit 2901234ad2954cd2388ced8d3aed2ed9de42803a.

[5] Jialong Xu, Bo Ai, Wei Chen, Ang Yang, Peng Sun, and Miguel Rodrigues. Wireless image transmission using deep source channel coding with attention modules. *IEEE Transactions on Circuits and Systems for Video Technology*, 32(4): 2315–2328, 2022.

[6] Ke Yang, Sixian Wang, Jincheng Dai, Kailin Tan, Kai Niu, and Ping Zhang. WITT: A wireless image transmission transformer for semantic communications. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2023.

[7] Ke Yang, Sixian Wang, Jincheng Dai, Xiaoqi Qin, Kai Niu, and Ping Zhang. SwinJSCC: Taming swin transformer for deep joint source-channel coding. *IEEE Transactions on Cognitive Communications and Networking*, 11(1):90–104, 2025.

[8] Ruofeng Yang, Yongcan Li, and Shuai Li. ARIS: Autonomous research via adversarial multi-agent collaboration. arXiv:2605.03042, technical report, 2026.

A. Reproducibility and AI-Assisted Workflow

Source and execution. The author WITT implementation is pinned to commit `2460db5` at https://github.com/KeYang8/WITT. Evaluation records identify the full commit and original source status; deployment files also retain source metadata. Entry points are `scripts/train_witt.py` (training), `scripts/prepare_gate_tables.py` (compilation), `scripts/evaluate_gate_tables_paired.py` (evaluation), and `scripts/analyze_paired_gate_results.py` (analysis). Plot generators read saved JSON results and require no inference. Code, raw metrics, figures, environment specifications, and a reproduction guide are available at https://github.com/RobinLuFae/aris-study-test.

Saved evidence. The selected checkpoint is identified by `R003-checkpoint-selection.json`. P002's preparation record supplies parameter counts and complete file sizes. P002 and P003 retain validation metrics; P004 retains per-method, per-noise, per-batch error sums and counts on the official test images. The test-freeze record precedes testing. Source snapshots accompanying P002–P004 preserve the evaluator and protocol. Original dataset pixels are separate from these source snapshots.

Actual AI use. The project uses ARIS's skill-based research workflow and evidence-review methodology [4, 8], with Codex as the executing AI. Our recorded workflow covers baseline selection, author-code inspection, resumable local training, candidate generation, critical review, method refinement, gated evaluation, and evidence review. Requested model identifiers and raw review replies are recorded. The reviews use the executor's model family, so their assessments remain provisional.

Pilot review corrected a subset-based maximum to the full-grid three-node loss of 0.093756 dB and motivated denser fractional-SNR validation. Code review required a separate synthetic reload check and verification of deployment sizes and checkpoint identity before testing. The process record and complete review replies accompany the code. This AI-assisted workflow supports the recorded experiments; individual student reflections are separate from the scientific results.